



Lab 3 : On-board FPGA Acceleration of AlexNet Convolution Layer for Classification Tasks with Vitis→Vivado→PYNQ Framework Flow Tutorial I : Unoptimized Matrix Multiplication

Assigned: Thursday, October 1st 2026
Due date: Thursday, October 8th 2026, 11:59pm

Contents

1	Motivation	1
2	Tutorial I : On-board FPGA Acceleration of Case Study I (Un-optimized Matrix Multiplication)	2
3	Convolution Layer in AlexNet Architecture for Image Classification	2
3.1	Image Classification	2
3.2	AlexNet Architecture	3
3.3	Convolution Layer #1	3
4	Your Tasks	3
4.1	Part 1 : FPGA Acceleration of the Optimized Matrix Multiplication Program . . .	4
4.1.1	Code Organization	4
4.1.2	To Do	4
4.1.3	Your Task	4
4.2	Part 2: Implementation of the first convolution layer in AlexNet Architecture	4
4.2.1	Code Organization	4
4.2.2	To Do	5
4.2.3	Your Task	5
5	What to Submit?	5

1 Motivation

In Lab 2, you studied how to write a HLS code and optimize it using the pragmas. Now, it is time to build your HLS IP (Intellectual Property), and run it on a real-FPGA. We are going to run our HLS designs on PYNQ-Z2 FPGA board, that supports PYNQ framework flow. The main goal of this lab is to get familiar with the **Vitis→Vivado→PYNQ Framework Flow**.

2 Tutorial I : On-board FPGA Acceleration of Case Study I (Un-optimized Matrix Multiplication)

Follow tutorial in zip folder, where you will learn how to build the HLS IP, invoke Xilinx Vivado and generate the bitstream and the hardware handoff files, which eventually will be stored on the FPGA to help test the IP via Jupyter Notebook.

3 Convolution Layer in AlexNet Architecture for Image Classification

3.1 Image Classification

Imagine I show you the image in Figure 1 and ask if you can recognize what's in it? It seems simple enough, right? For humans, categorizing the objects, and identifying features in an image is usually an effortless task. However, for a computer, reaching the same conclusion demands significant computational power and a carefully designed, well-trained deep learning model. This challenge applies not only during the training process but also when making predictions during inference.



Figure 1: Golden Fish example from ImageNet1k Dataset

Image classification is a computer vision task that involves assigning a label or category to an image based on its content, aiming to identify and categorize the objects, scenes, or features present in the image. This task is commonly performed using machine learning models, particularly deep learning architectures like Convolutional Neural Networks (CNNs), which learn to recognize patterns in images through training on labeled datasets. The process begins with data collection, where a large dataset of labeled images is gathered, followed by preprocessing to normalize and prepare the images for the model. The model is then trained on this dataset, after which its performance is evaluated on a separate test set to assess accuracy. Once trained, the model can classify new, unseen images. Image classification has a wide range of applications, including object recognition in photos and videos, medical imaging for disease diagnosis, autonomous vehicles recognizing road signs and obstacles, and content moderation on social media platforms.



3.2 AlexNet Architecture

AlexNet is a deep learning architecture designed for image classification tasks, and it gained significant attention after winning the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) in 2012. AlexNet consists of eight layers: five convolutional layers followed by three fully connected layers. The structure is designed to extract hierarchical features from images and progressively learn complex patterns. In addition, AlexNet was one of the first architectures to effectively utilize Graphics Processing Units (GPUs) for training deep networks, significantly speeding up the training process.

3.3 Convolution Layer #1

Layer Parameters	Symbol	Layer Values
Input Depth	M	3
Input Width/Height	I	224
Num of Kernels	F	64
Kernel Width/Height	K	11
Output Depth	N	64
Output Width/Height	O	55
Stride	S	4
Padding	P	2

Table 1: First Convolution Layer Parameters

We are interested in implementing this first convolution layer of AlexNet using Figure .1 1 as the input image. This first convolution layer takes an input image (feature map) of size $M \times I \times I$ (e.g., $3 \times 224 \times 224$), and convolve it with a filter F of size 64 kernels, each of dimensions $(F \times M \times K \times K)$ where K is the kernel height/width. The resulting output feature is of size of size $N \times O \times O$ (e.g., $64 \times 55 \times 55$). The output feature map is calculated using

$$O = ((I - K + 2 \times P) \div S) + 1 \quad (1)$$

The convolution operation essentially performs a dot product operation between the filters and the local regions of the input. A common implementation pattern of the CONV layer is to take advantage of this fact and formulate the forward pass (inference) of the convolutional layer as one big matrix multiply.

4 Your Tasks

Before you start working on lab3 tasks, download and unzip the file named lab3.zip to your home directory on hunmin server. Be sure to source the class setup script using the following command every time you login to your account.

```
$ source /import/scripts/ece588.cshrc
```



4.1 Part 1 : FPGA Acceleration of the Optimized Matrix Multiplication Program

4.1.1 Code Organization

You are given a zip file named lab3.zip on the Canvas site, navigate to part1 folder which contains the following files for you to build tutorial I and part 1 project.

- **part1/matmult.hpp**: the header file of the matrix multiplication program.
- **part/matmult.cpp**: the **complete unoptimized synthesizable** file of the matrix multiplication program.
- **part1/tb_matmult.cpp**: a test bench file that helps verify the code.
- **vitis_hls.tcl**: a script file to automatically build the HLS IP, and specify the FPGA device and the clock cycle we are targeting. **We are targeting the Pynq-Z2 FPGA board with a clock cycle of 10ns.**
- **matmul.ipynb**: Jupyter Notebook
- **part1/Makefile**: a Makefile for you to easily :
 - (1) simulate your program : just type **make csim**
 - (2) synthesize and build the HLS IP : just type **make ip**
 - (3) clean the project : just type **make clean**

4.1.2 To Do

1. Follow tutorial I, and run the unoptimized HLS IP on FPGA to report your findings
2. Now take your optimized code you worked on in Lab2 and accelerate it on FPGA. (This should be very straight forward by following Tutorial I and report your findings)

4.1.3 Your Task

Your main task is to accelerate your optimized Matrix Multiplication program you worked on in lab2 on PYNQ-Z2 FPGA, and report your findings

4.2 Part 2: Implementation of the first convolution layer in AlexNet Architecture

4.2.1 Code Organization

1. **part2/params**: folder that contains the reference binary files
2. **part2/conv.hpp**: is the header file
3. **part2/conv.cpp**: is the source code file that **you need to update and implement the 11 x 11 convolution**
4. **part2/tb_conv.cpp**: is the testbench that checks for functional correctness



5. **part2/vitis_hls.tcl**: a script file to automatically build the HLS IP, and specify the FPGA device and the clock cycle we are targeting. **We are targeting the Pynq-Z2 FPGA board with a clock cycle of 10ns.**
6. **part2/Makefile**: a Makefile for you to easily :
 - (1) simulate your program : just type **make csim**
 - (2) synthesize and build the HLS IP : just type **make ip**
 - (3) clean the project : just type **make clean**

4.2.2 To Do

1. Implement a functionally correct 11x11 convolution layer.
2. Verification: run your program by typing **make csim** to make sure the program simulate without any errors. (Ignore warnings!).
3. Run **make ip** to build your unoptimized HLS IP
4. Generate the bitstream and the hardware handoff files.
5. Accelerate your unoptimized and optimized HLS IP Conv on PYNQ-Z2 FPGA.
6. You will have to write the Jupyter Notebook file.

4.2.3 Your Task

1. Your first task is to complete the code in **conv.cpp** file. You have to write a C/C++ program that performs 11 x 11 convolution. You are provided with a testbench code which reads the input image, layer parameters, as well the reference output feature map and performs a point-to-point comparison with the obtained feature map. The Mean Squared Error (MSE) function is used as the metric for evaluation. Note: the input feature map is already padded and ready to get convolved with the kernel.
2. Accelerate your unoptimized convolution program using the Vitis→Vivado→PYNQ Framework Flow and report your findings **You will have to write your own Notebook**
3. Now, optimize your convolution layer with pragmas and accelerate it on the board and report your findings **YES I KNOW, THIS IS NOT AN EASY TASK!**

5 What to Submit?

- Results of running your unoptimized matrix multiplication HLS IP on the FPGA board: **10 points**
 1. matmul.bit (Bitstream) 3 points
 2. matmul.hwh (Hardware Handoff file) 3 points
 3. matmul.ipynb (Jupyter Notebook) 4 points
- Results of running your Optimized matrix multiplication HLS IP on the FPGA board: **15 points**



-
1. matmul.bit (Bitstream) 5 points
 2. matmul.hwh (Hardware Handoff file) 5 points
 3. matmul.ipynb (Jupyter Notebook) 5 points
- Discuss the below questions in your report: 15 points
 1. compare between the Vitis HLS latency estimate (report that was generated by the tool) vs the actual latency achieved on-board for unoptimized and optimized IPs. 5 points
 2. compare between the speed-up achieved from running the HLS IP on PL vs PS latency for unoptimized and optimized IPs. 5 points
 3. Why is the on-board latency different than the Vitis estimate for unoptimized and optimized IPs? 5 points
- Results of running the unoptimized convolution layer on-board: 20 points
 1. conv.cpp (source code) 5 points
 2. conv.bit (Bitstream) 5 points
 3. conv.hwh (Hardware Handoff file) 5 points
 4. conv.ipynb (Your Jupyter Notebook) 5 points
 - Results of running the optimized convolution layer on-board: 20 points
 1. conv.cpp (source code) 5 points
 2. conv.bit (Bitstream) 5 points
 3. conv.hwh (Hardware Handoff file) 5 points
 4. conv.ipynb (Your Jupyter Notebook) 5 points
 - Discuss the below questions in your report: 20 points
 1. compare between the Vitis HLS latency estimate (report that was generated by the tool) vs the actual latency achieved on-board for unoptimized and optimized IPs. 5 points
 2. compare between the speed-up achieved from running the HLS IP on PL vs PS latency for unoptimized and optimized IPs 5 points
 3. Why is the on-board latency different than the Vitis estimate for unoptimized and optimized IPs? 10 points

NOTE : PROVIDE ALL THE NECESSARY SCREENSHOTS TO SUPPORT YOUR CLAIM
